1. Composable Finance 프로젝트 구조 분석

1.1 프로젝트 개요

Composable Finance는 크로스체인 상호운용성과 탈중앙화된 블록 빌딩을 위한 인프라를 구축하는 복합적인 블록체인 프로젝트입니다. 이 프로젝트는 Substrate 프레임워크를 기반으로 하며, Polkadot 파라체인(Composable)과 Kusama 파라체인(Picasso)을 포함합니다.

핵심 기술 스택

- **Substrate Framework**: 블록체인 개발 프레임워크
- **Polkadot SDK**: 파라체인 구현을 위한 도구
- **Rust**: 주요 개발 언어
- **Nix**: 개발 환경 관리
- **CosmWasm**: 스마트 컨트랙트 실행 환경

1.2 최상위 디렉토리 구조

```
composable/
├── code/ # 핵심 블록체인 코드
├── docs/ # 문서화 시스템 (Docusaurus)
├── audits/ # 보안 감사 보고서
├── flake/ # Nix 개발 환경 설정
├── inputs/ # 외부 의존성 서브모듈
```

```
├── rfcs/ # RFC (Request for Comments) 문서
├── tests/ # 통합 테스트
├── flake.nix # Nix 플레이크 설정
├── README.md # 프로젝트 개요
└── RELEASE.md # 릴리스 프로세스 가이드
```

1.3 핵심 디렉토리 상세 분석

1.3.1 code/ 디렉토리 - 블록체인 핵심 구현

`code/` 디렉토리는 Composable Finance의 핵심 블록체인 구현을 포함합니다.

```
code/
├── parachain/ # 파라체인 구현
| ├── frame/ # 커스텀 팰릿들
| ├── runtime/ # 런타임 구성
| ├── node/ # 노드 구현
| └── src/ # 메인 바이너리
├── cvm/ # Cross-chain Virtual Machine
├── services/ # 외부 서비스들
├── utils/ # 유틸리티 도구들
├── integration-tests/ # 통합 테스트
└── Cargo.toml # 워크스페이스 설정
```

1.3.1.1 parachain/frame/ - 커스텀 팰릿 구현

Substrate 기반의 커스텀 팰릿들이 구현되어 있습니다:

DeFi 관련 팰릿:

- `pablo/` : DEX (Decentralized Exchange) 구현
- `liquid-staking/` : 유동성 스테이킹 기능
- `farming/` : 유동성 마이닝 및 인센티브
- `dex-router/` : DEX 라우팅 최적화

크로스체인 관련 팰릿:

- `pallet-multihop-xcm-ibc/` : XCM과 IBC를 결합한 멀티홉 전송
- `pallet-xcm-helper/` : XCM 헬퍼 유틸리티
- `revenue-ibc/` : IBC 기반 수익 분배

인프라 팰릿:

- `oracle/` : 가격 오라클 시스템 (Apollo)
- `assets/` : 자산 관리
- `assets-registry/` : 자산 레지스트리
- `cosmwasm/` : CosmWasm 스마트 컨트랙트 지원

거버넌스 및 유틸리티:

- `crowdloan-rewards/` : 크라우드론 리워드 관리
- `vesting/` : 토큰 베스팅
- `transaction-payment/` : 트랜잭션 수수료 처리

1.3.1.2 parachain/runtime/ - 런타임 구성

```
runtime/
├── composable/ # Composable 파라체인 런타임
```

```
├── picasso/ # Picasso 파라체인 런타임
├── common/ # 공통 런타임 구성요소
├── primitives/ # 런타임 프리미티브
├── composable-wasm/ # Composable WASM 런타임
└── picasso-wasm/ # Picasso WASM 런타임
```

각 런타임은 `construct_runtime!` 매크로를 사용하여 팰릿들을 통합합니다:

```rust
// composable/src/lib.rs 예시
construct_runtime!(
pub  enum Runtime where
Block = Block,
NodeBlock = primitives::Block,
UncheckedExtrinsic = UncheckedExtrinsic,
{
System: frame_system,
Pablo: pallet_pablo,
LiquidStaking: pallet_liquid_staking,
Oracle: pallet_oracle,
// ... 기타 팰릿들
}
);
```

1.3.1.3 parachain/node/ - 노드 구현

노드 서비스와 네트워킹 구현:

- 클라이언트 서비스 초기화
- P2P 네트워킹 설정
- RPC 서비스 구성
- 콜레이터 기능

1.3.2 docs/ 디렉토리 - 문서화 시스템

Docusaurus 기반의 문서화 시스템:

```
docs/
├── docs/ # 마크다운 문서들
├── src/ # React 컴포넌트
├── static/ # 정적 자산
├── docusaurus.config.js # Docusaurus 설정
└── package.json # Node.js 의존성
```

1.3.3 audits/ 디렉토리 - 보안 감사

보안 감사 보고서와 수정사항들:

```
audits/
├── halborn/ # Halborn 감사 보고서
├── mantis-contracts/ # Mantis 컨트랙트 감사
├── picasso-cosmos/ # Picasso Cosmos 감사
├── solana-bridge-contract/ # Solana 브릿지 감사
```

```
└── README.md # 감사 개요
```

1.3.4 flake/ 디렉토리 - Nix 개발 환경

Nix 기반 개발 환경 설정:

```
flake/
├── dev-shells.nix # 개발 셸 환경
├── rust.nix # Rust 툴체인 설정
├── cosmos.nix # Cosmos 관련 도구
├── ethereum.nix # Ethereum 도구
└── devnets/ # 개발 네트워크 설정
```

1.4 Cargo 워크스페이스 구성

1.4.1 워크스페이스 멤버

`code/Cargo.toml` 에서 정의된 워크스페이스 구조:

```
[workspace]
members = [
"parachain/frame/*", # 모든 팰릿들
"parachain/node", # 노드 구현
"parachain/runtime/*", # 모든 런타임들
"services/cmc-api", # CMC API 서비스
```

```
"utils/collator-sidecar", # 콜레이터 사이드카
"utils/common", # 공통 유틸리티
"utils/price-feed", # 가격 피드 도구
"cvm/lib/core/", # CVM 코어
]
```

1.4.2 주요 의존성 관계

Substrate/Polkadot 의존성:

- `polkadot-v0.9.43` 브랜치 사용
- Cumulus 파라체인 SDK
- XCM (Cross-Consensus Messaging)

외부 라이브러리:

- ORML (Open Runtime Module Library)
- IBC (Inter-Blockchain Communication)
- CosmWasm 런타임

내부 의존성:

```
// Pablo DEX 팰릿 예시
use composable_traits::{
dex::{Amm, BasicPoolInfo, Fee},
assets::CreateAsset,
};
use composable_support::math::safe::SafeArithmetic;
```

1.5 빌드 및 컴파일 구성

1.5.1 프로파일 설정

```
[profile.release]
# Polkadot 런타임은 unwinding이 필요
lto = false
panic = "unwind"

[profile.production]
inherits = "release"
lto = true

[profile.cosmwasm-contracts]
codegen-units = 1
opt-level = "s"
panic = "abort"
```

1.5.2 바이너리 타겟

```
[[bin]]
name = "composable"
path = "parachain/src/main.rs"
```

메인 바이너리는 `parachain/src/main.rs` 에서 시작되며, 다음과 같은 기능을 제공합니다:

- Composable 파라체인 노드 실행
- Picasso 파라체인 노드 실행
- 개발 모드 및 테스트넷 지원

1.6 개발 도구 및 설정

1.6.1 코드 품질 도구

- **Clippy**: Rust 린터 설정 (`clippy.toml`)
- **Rustfmt**: 코드 포맷터 설정 (`rustfmt.toml`)
- **Deny**: 의존성 보안 검사 (`deny.toml`)

1.6.2 Nix 통합

프로젝트는 Nix를 사용하여 재현 가능한 개발 환경을 제공:

```
# flake.nix에서
{
inputs = {
nixpkgs.url = "github:NixOS/nixpkgs/nixos-unstable";
rust-overlay.url = "github:oxalica/rust-overlay";
};
outputs = { self, nixpkgs, rust-overlay }: {
devShells = {
# 개발 셸 환경들
```

```
};

};

}
```

1.7 테스트 구조

1.7.1 단위 테스트

각 팰릿은 `#[cfg(test)]` 모듈을 포함하여 단위 테스트를 제공합니다.

1.7.2 통합 테스트

`integration-tests/` 디렉토리에서 크로스 팰릿 기능을 테스트합니다.

1.7.3 벤치마크

성능 측정을 위한 벤치마크 테스트가 각 팰릿에 포함되어 있습니다.

1.8 배포 및 릴리스

1.8.1 릴리스 프로세스

`RELEASE.md` 에 정의된 체계적인 릴리스 프로세스를 따릅니다.

1.8.2 GitHub Actions

`.github/workflows/` 에서 CI/CD 파이프라인을 관리합니다.

1.9 외부 통합

1.9.1 서브모듈 의존성

`inputs/` 디렉토리에서 외부 프로젝트들을 서브모듈로 관리:

- AcalaNetwork
- bifrost-finance
- paritytech
- wynddao

1.9.2 크로스체인 연결

- **IBC**: Cosmos 생태계와의 연결
- **XCM**: Polkadot 생태계 내 통신
- **Solana**: Solana 블록체인과의 브릿지

이 구조 분석을 통해 Composable Finance 프로젝트가 어떻게 모듈화되고 조직화되어 있는지, 그리고 각 컴포넌트가 어떤 역할을 하는지 이해할 수 있습니다.